# Chapter 1

## INTRODUCTION

## 1.1 Introduction

Accounting software describes a type of application software that records and processes accounting transactions within functional modules such as accounts payable, accounts receivable, journal, general ledger, payroll, and trial balance. It functions as an accounting information system. Accounting software may be on-line based, accessed anywhere at any time with any device which is Internet enabled, or may be desktop based. It varies greatly in its complexity and cost. A financial accounts package keeps track of an organization's finances by recording and processing transactions within a business.

At the centre of such systems, there is usually a general ledger, a facility for handling accounts payable and receivable, a payroll module, and a reporting function.

Accounting Software is vital for a business to fully understand their situation, determine the success of their business performance, and run a successful venture.

We can write checks, create bank deposits, and prepare customer invoices in the solution, and the software will record all the transactions.

In manual accounting, most of the work is done at the end of the financial year when the records are prepared for administration and tax assessment purposes. But with computerized bookkeeping, we can create and maintain financial records continuously and get in-depth reports any time you wish. This enables company management to continuously monitor the financial records of the business and resolve issues early before they balloon into big problems.

Manual accounting increases the chances of errors in calculation. We can avoid this by opting for accounting software which automatically performs all calculations accurately. This is a big benefit especially if you need to prepare detailed invoices in which specific items or worked hours are billed at various rates.

Manual invoices can easily be misplaced or lost. On the other hand, an accounting software stores all the data and information securely in a single location. We can review, reprint, and resend invoices if needed and we can easily search the software for invoices with invoice numbers, names, and amounts.

So accounting management systems software offers a range of useful features and companies usually select a solution that has an attractive interface and is easy to use.

## 1.2 Objective

Accounting management system in generally operated manually. The manual system is time consuming, unsecured, less reliable and expensive. The manual Accounting management system is completely people dependent process. So it's very difficult to manage accounts system, create balance sheet, daily reports, income statement, trial balance, chart of accounts, financial year closing etc. Now a day's tool is very essential for accounting management which is easy-to-use, comprehensive and affordable system to handle all the present and future accounting related operational needs. This software will eliminate all those aforementioned problems.

We are believed, this software those people will not have to work so long and the decision making will be easier and faster.

## 1.3 Motivation

This project is to increase working flow or accounts management flow and minimizes time consuming, mistakes and continuous financial monitoring and centralizes key business processes and tasks such as customer relationship management, payroll, invoicing, inventory management and more. In computerized bookkeeping, we can create and maintain financial records continuously and get reports any time when we've need. which is easy-to-use, comprehensive and affordable system to handle all the present and future accounting related operational needs.

## 1.4 Tools

### 1.4.1 Hardware:

- PROCESSOR : Intel Dual Core
- RAM : 1 GB
- HARD DISK : 500 GB

### 1.4.2 Software:

- XAMPP server environment
- Dreamweaver
- Mysql database
- MVC Codeigniter Framework

### 1.4.3 Technology:

**Front-end:**

- ♦ HTML5
- ♦ CSS3
- ♦ Bootstrap Framework v3.4
- ♦ JavaScript
- ♦ JQuery
- ♦ Responsive template

**Back-end:**

- ♦ Object Oriented PHP
- ♦ Codeigniter Framework v3.1
- ♦ MySQL database
- ♦ XAMPP (development environment)

## 1.5 Methodology

Methodology is a collection of procedure, techniques, tools and documentation aids that help the system development in their effort to implement a new information system. The proposed approach is wanted to meet the goal of doing all voluntary works.

# Chapter 2

## PROBLEM STATEMENT

## 2.1 Problem Statement

The main goal of this project is to create a fully functioning Voluntary organization solution ready to be deployed on the voluntary community:

- ✓ Get a general understanding of a Voluntary organization management solution.
- ✓ Get a general understanding of Template Mastering.
- ✓ Get a general understanding of HTML,BOOTSTARP & JavaScript.
- ✓ Get a general understanding of Codeigniter (PHP framework) and MySQL.
- ✓ Get a general understanding of membership, donation, job assignment and voluntary works record.
- ✓ Discuss the result of the proposed work.

# Chapter 3

## LITERATURE REVIEW

## 3.1 Project Life Cycle

The project life cycle includes various development phases that occur in the life of project starting right from the inception of the project to its final development at the member's end. The three development phases in a project life cycle are:

- Project initiation
- Project execution
- Project deployment

### 3.1.1 Project initiation:

The project initiation phase is first phase of life cycle. This phase involves creating a complete
plan for the project, specifying various activities that will be performed and assigning responsibilities to team members on the basis of their skill set.

### 3.1.2 Project execution:

After the project plan is made and the responsibilities assigned, the actual development of the project starts. The phase in which the actual development of the project takes place is known as the project execution phase. This is the most crucial phase of any project and is subdivided into the following phases:

1. ***System analysis***
    - Initial Study
    - Information Gathering
    - Feasibility Study
2. ***System Design***
    - Design standard
    - High level design & design tools
    - Database design
    - Logical design
    - Construction

3. ***System implementation***
    - Combination and testing
    - Post execution

### 3.1.3 Project development:

After the undertaking execution stage, the last period of a venture life cycle is the task advancement stage. In this stage, the conveyed at the member side. This stage additionally includes giving client backing to the member for some predefined time frame. At the point when undertaking is manufactured it might potentially remain blunder less of additional, in light of the fact that few sort of alteration can happen a few times. So for the first
run through when we run the database site we discovered couple of issues in instruments segments. We altered this issue including some minor issues quickly, and subsequently the application runs legitimately.

# Chapter 4

**Project Modules & System Analysis**

## 4.1 Project Modules

There are two kind of project module

- Sub System Module
- Application Module

### 4.1.1 Sub system module:

#### 4.1.1.1 User Management:

- ✓ Create Users:
- ✓ Remove Users
- ✓ Update User information
- ✓ Access permission

#### 4.1.1.2 Chart of Accounts:

- ✓ Create Accounts Head
- ✓ Update Accounts Head
- ✓ Remove Accounts Head

#### 4.1.1.3 Transaction:

- ✓ Pay Bill
- ✓ Receive Bill
- ✓ Show Voucher

#### 4.1.1.4 Report:

- ✓ Account Ledger
- ✓ Trial Balance
- ✓ Income Statement
- ✓ Balance Sheet
- ✓ Owner's Equity

### 4.1.2 Application Module:

#### 4.1.2.1 Admin Module:

- ✓ Can create user
- ✓ Can see all users
- ✓ Can update any users information
- ✓ Can set users access in this software
- ✓ Can see all access for every users
- ✓ Can set company information
- ✓ Can set voucher type
- ✓ Can open/close day accounts
- ✓ Can close yearly financial accounts
- ✓ Can create/check/approve vouchers
- ✓ Can see any kind of reports

#### 4.1.2.2 User Module:

- ✓ Can do anything in this software depends on access
- ✓ With access can create/check/approve vouchers
- ✓ With access can see reports

## 4.2 Process Modeling

Process modeling involves graphical representing the functions, or process, which capture, manipulate, store, and distributed between a system and its environment and between components within a system.

## 4.3 Input Design

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data into a usable form for processing data entry.

## 4.4 Out Design

Computer output is the most important and direct information source to the user. Output design is a process that involves designing necessary outputs in the form of reports that should be given to the users according to the requirements. Efficient, intelligible output design should improve the system's relationship with the user and help in decision making.

Since the reports are directing referred by the management for taking decision and to draw conclusions they must be designed with almost care and the details in the reports must be simple, descriptive and clear to the user.

## 4.5 System Analysis

### 4.5.1 Existing System

The existing system analysis is the first step for requirement engineering. Currently, the existing system is runs manually.

The major drawbacks of the existing system are given below

- ✓ Time consuming, tedious and less efficient.
- ✓ Maintenance cost is higher.
- ✓ System is greatly exposed to human error.
- ✓ More number of employees needed.
- ✓ Updating any information is costly and error tedious because it requires checking of multiple files.

### 4.5.2 Proposed System

Our proposed system is to develop an automated Accounting management system by applying the software engineering principles and technique.
The following functions for the proposed system have been established. According to the specified requirements of the system, the software will perform these functions:

- ✓ Fill and save company information.
- ✓ Update the company information.
- ✓ Add, delete, and update chart of accounts information.
- ✓ Add, update and delete voucher type information.
- ✓ Add, delete, and edit user information.
- ✓ Add, update user access.
- ✓ Add, check and approve voucher information.

## 4.5.3 Flow Chart of Proposed System

Our projects in a particular style shown are fig.2.3. When a new entry is occurred in the system, it first checks whether the user is valid or not. If the user is valid he/she can enter data into the system.

#### 4.5.3.1 Flow Chart of new entry form

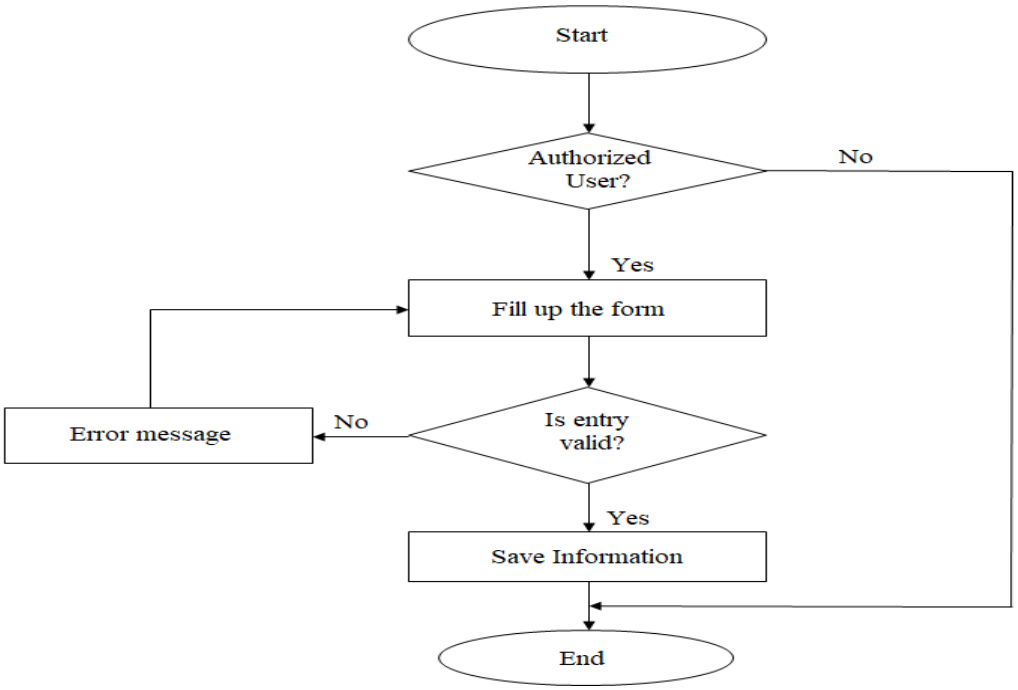



Figure 4.1: Flow chart of New Entry Form (Proposed System)

**4.5.3.2 Flow chart on edit entry form**.

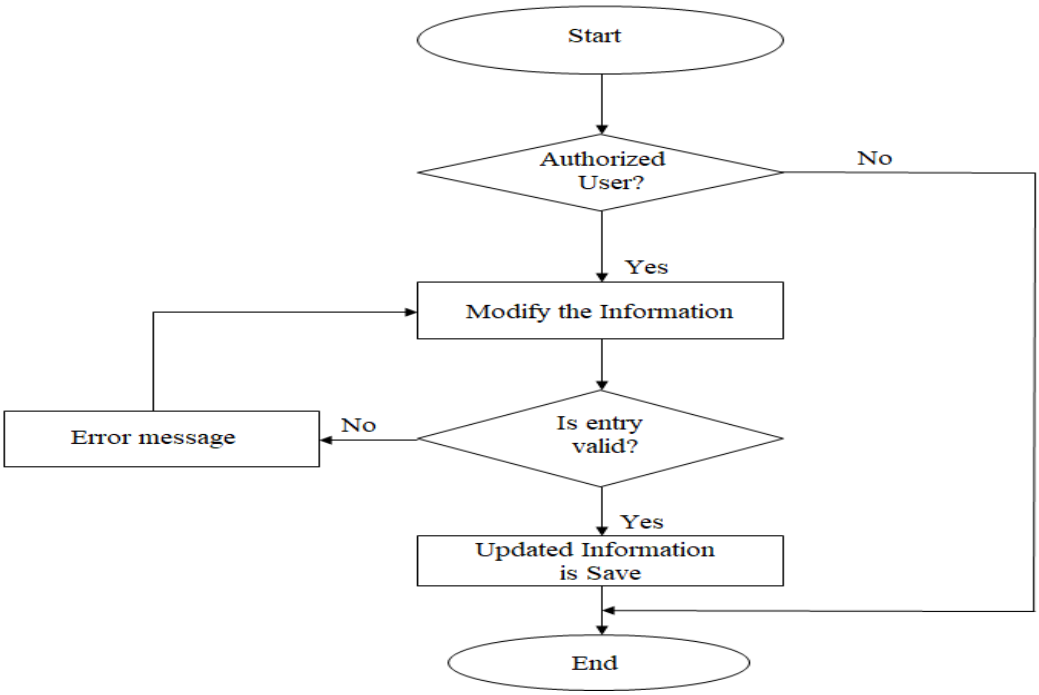



Figure 4.2: Flow chart of Edit Entry Form (Proposed System)

# Chapter 5

**System Design**

## 5.1 Introduction

This chapter includes DFD (Data Flow Diagram), ERD (Entity Relationship Diagram) of proposed system. It include data dictionary of proposed system. It covered the hardware specification and design of proposed system. And it covers the system tools used in this proposed system.

## 5.2 Development Model

When developing the software, a software development model has to be chosen. It works as a guidance and defines different phases of the software development which makes it the development manageable and traceable. There are several popular models as waterfall model. The waterfall model is followed in this project.

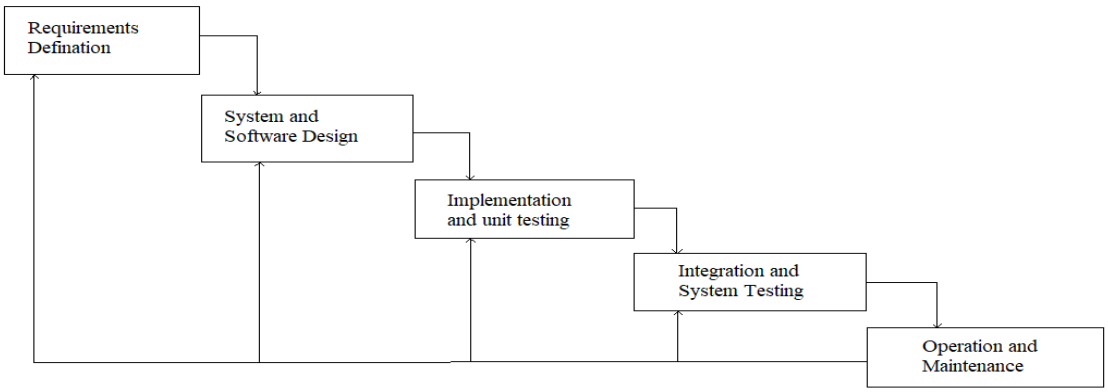



Figure 5.1: Waterfall Model.

## 5.3 Data Flow Diagram

Data flow diagram will act as a graphical representation of the system in terms of interaction between the system, external entities, and process and how data stored in certain location.

There are four kinds of system components:

- External entities:
  - The objects outside the system, with which the system communicates.
- Data stores:
  - They are the repositories in the system, also referred to as files.
- Process
  - It transforms the incoming data flow into outgoing data flow.
- Data flow
  - They are pipelines through which packets of information flow. Label the arrows with the name of the data that moves through it.

### 5.3.1 Context diagram of Accounting software using System:

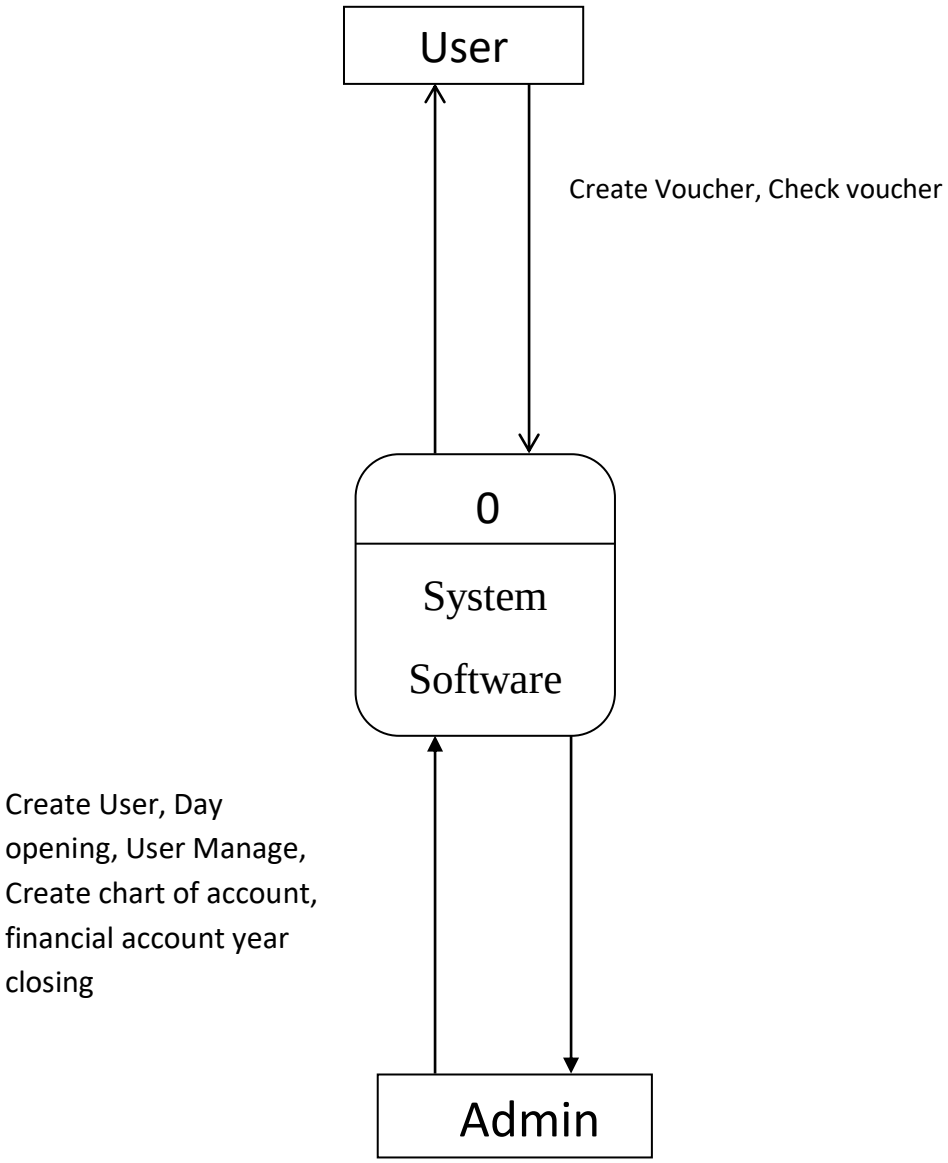



Fig: 5.3.1: Accounting software Data Flow Diagram

5.3.2: **Context diagram of accounting software user login diagram:**

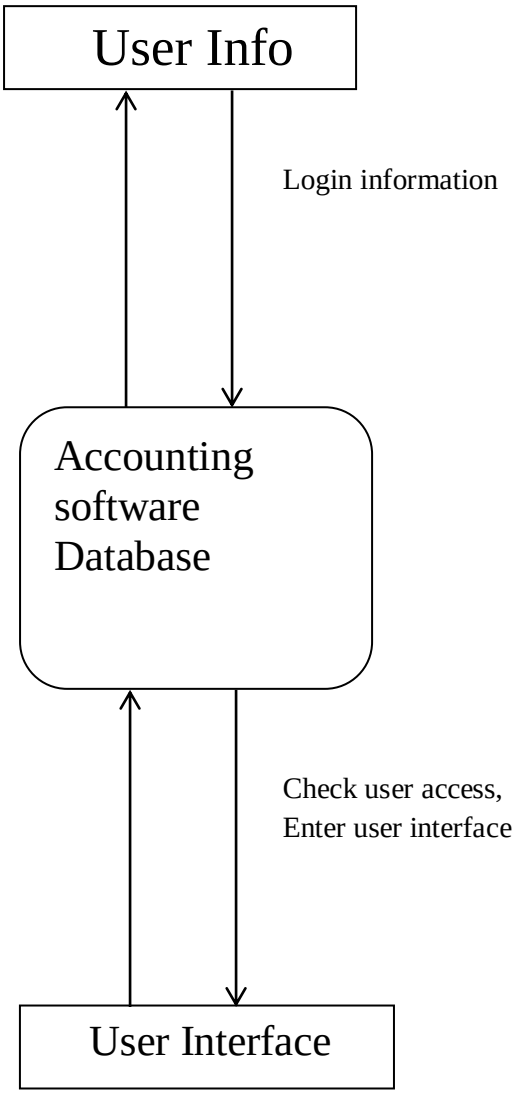



Fig: 5.3.2: Accounting software User Login Data Flow Diagram.

5.3.3: **Context diagram of accounting software user management:**

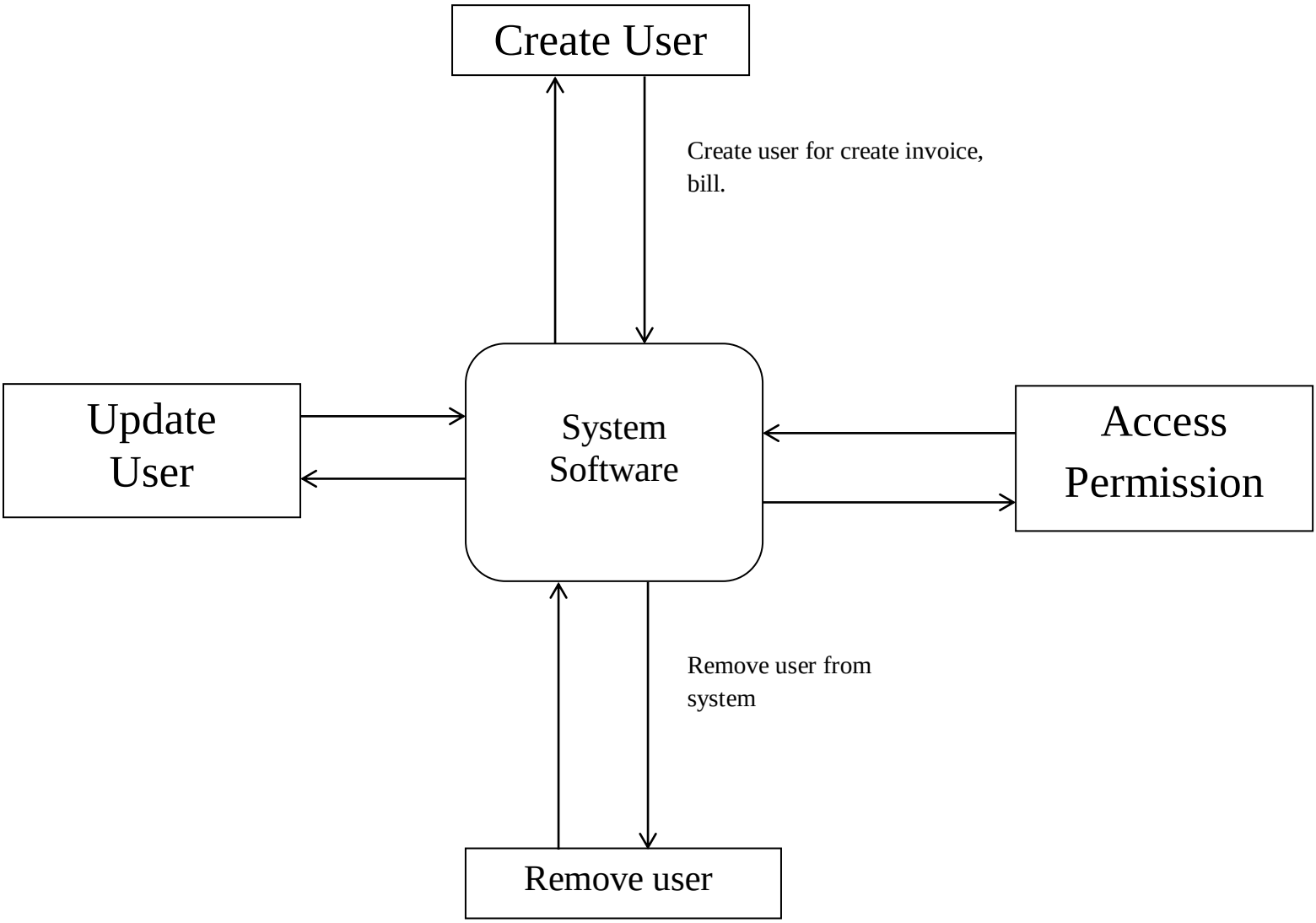



Fig: 5.3.3: Accounting software user management Data Flow Diagram.

5.3.4: **Context diagram of accounting software Transaction:**

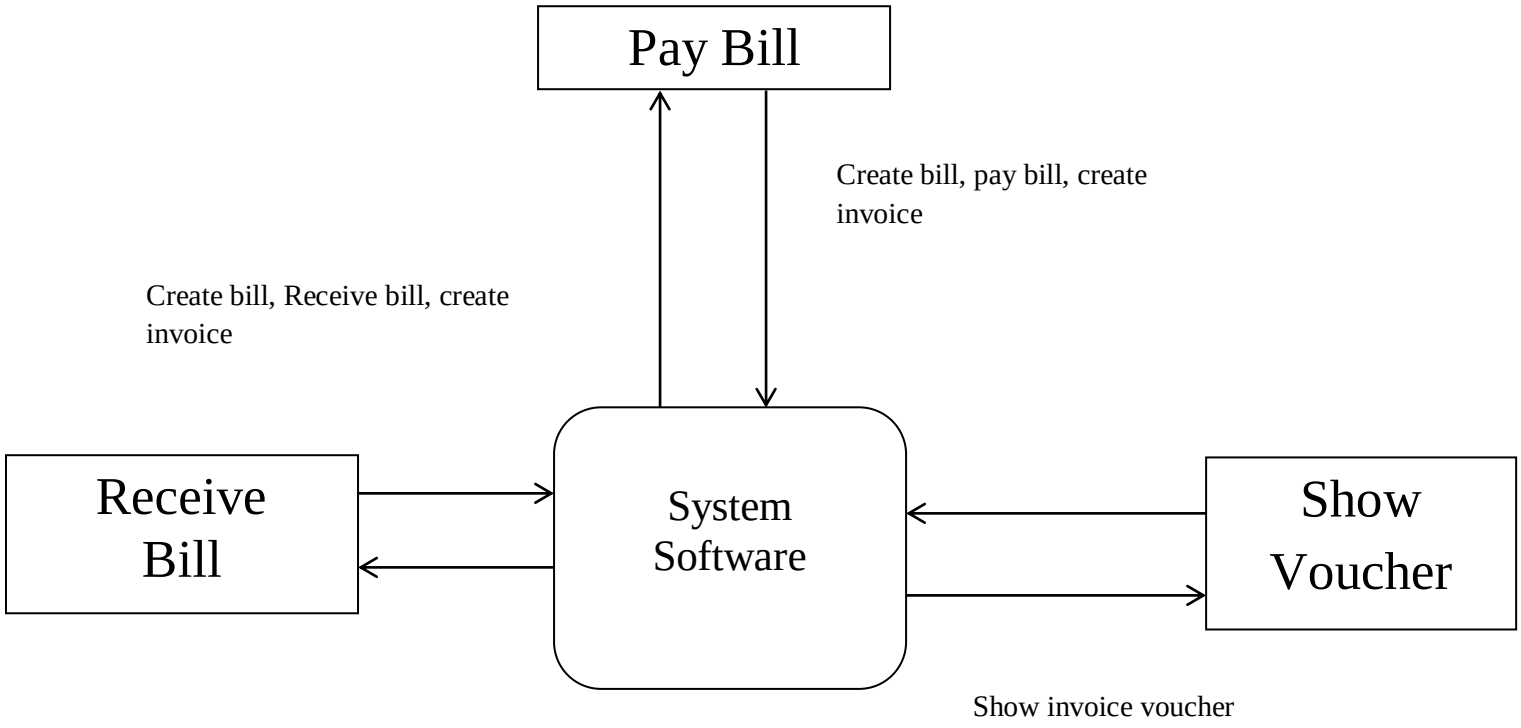



Fig: 5.3.4: Accounting software Transaction Data Flow Diagram.

## 5.4 Entity Relationship Model

An entity-relationship model is a graphical representation of entries and their relationships to each other [3].

For a binary relationship set *R* between entity sets *A* and *B*, the mapping cardinality must be one of the following:

- ✓ **One-to-one:** An entity in A is associated with at most one entity in B, and an entity in B is associated with at most one entity in A. [3].

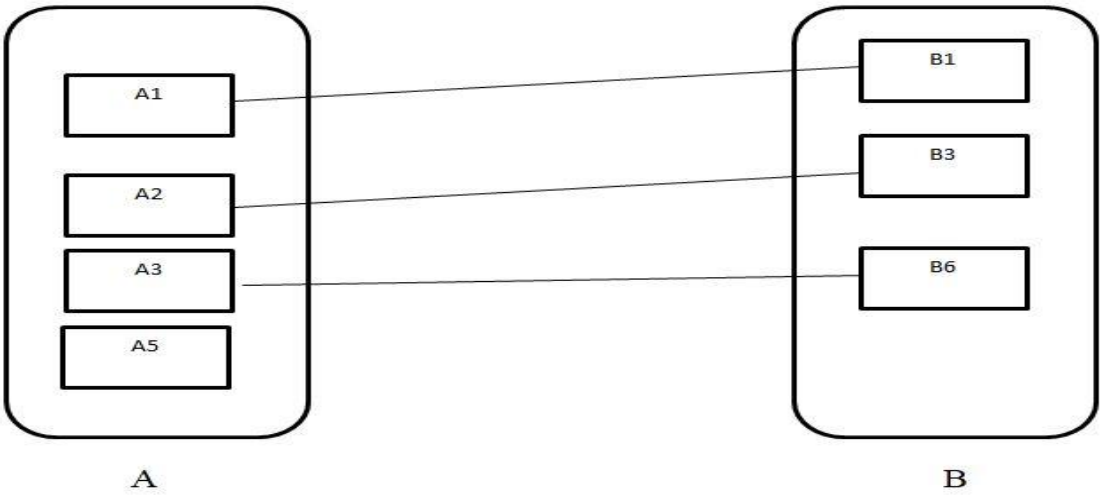



Figure 5.4.1: One to one entity.

- ✓ **One-to-many:** An entity in A is associated with any number (zero or more) of entities in B. An entity in B, however, can be associated with at most one entity in A. [3].

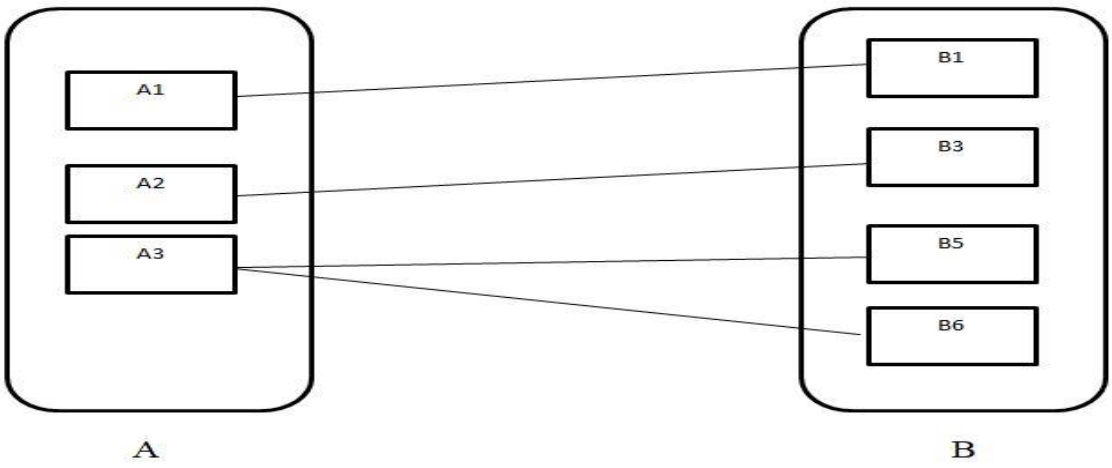



Figure 5.4.2: One to many entities.

- ✓ **Many-to-one:** An entity in A is associated with at most one entity in B. An entity in B, however, can be associated with any number (zero or more) of entities in A .[3]

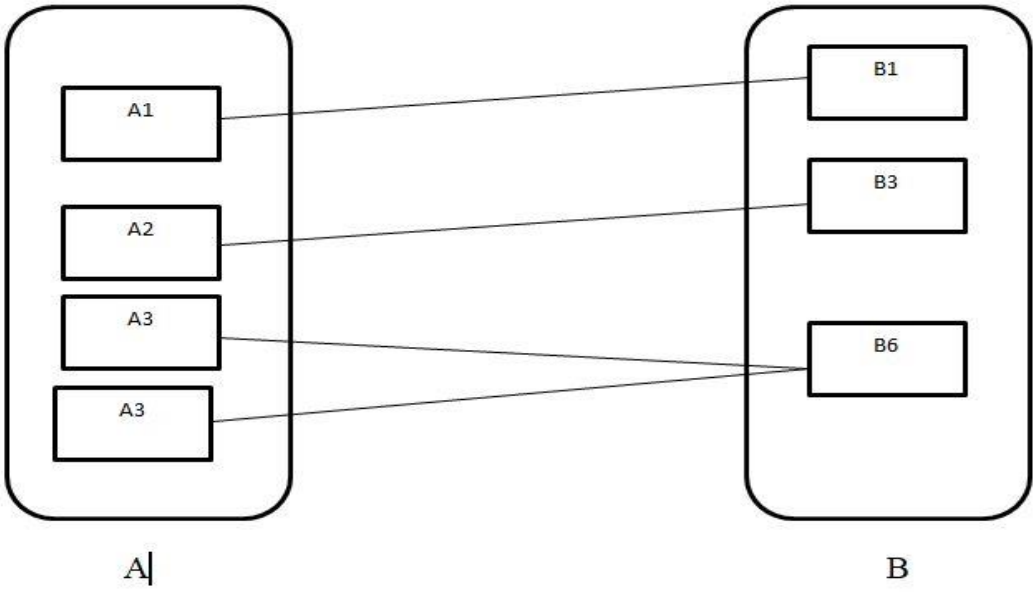



Figure 5.4.3: Many to one entity.

- ✓ **Many-to-many:** An entity in A is associated with any number (zero or more) of entities in A. [3]

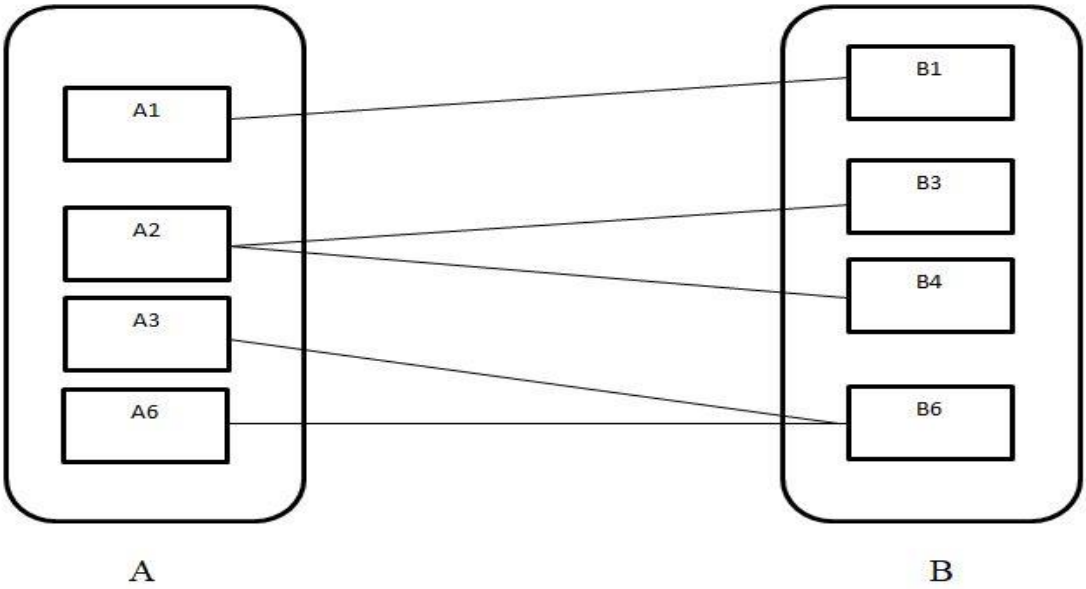



Figure 5.4.4: many to many entities.

## 5.5 Schema Diagram

This is an overview of all the flowchart symbols that you will use when drawing flowcharts and process flow. All these objects are available in create and we can try out a demo or take a look at our sample flowchart for more context.

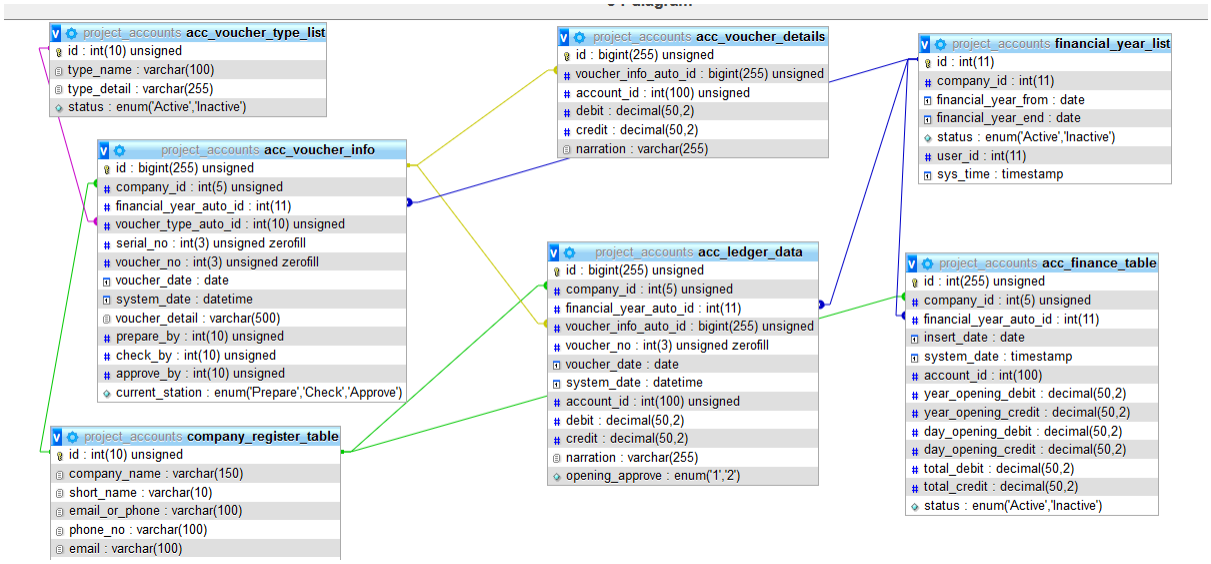



**Fig 5.5: Schema Diagram**

## 5.6 Code Optimization

Code optimization aims at improving execution efficiency of a program. This is achieved in two ways:

- ✓ Redundancies in a program are eliminated.
- ✓ Computations in a program are rearranged or rewritten to make it execute efficiently.

The optimization must not change the meaning of a program. The accounting management system optimizes the code by using the optimization techniques such as dead code elimination and frequency reduction. Thus improves the execution efficiency.

.

# Chapter 6

**Project Layout**

## 6.1 Dashboard:

When Users use this Accounting management software they see first the dashboard of software (fig 6.1).

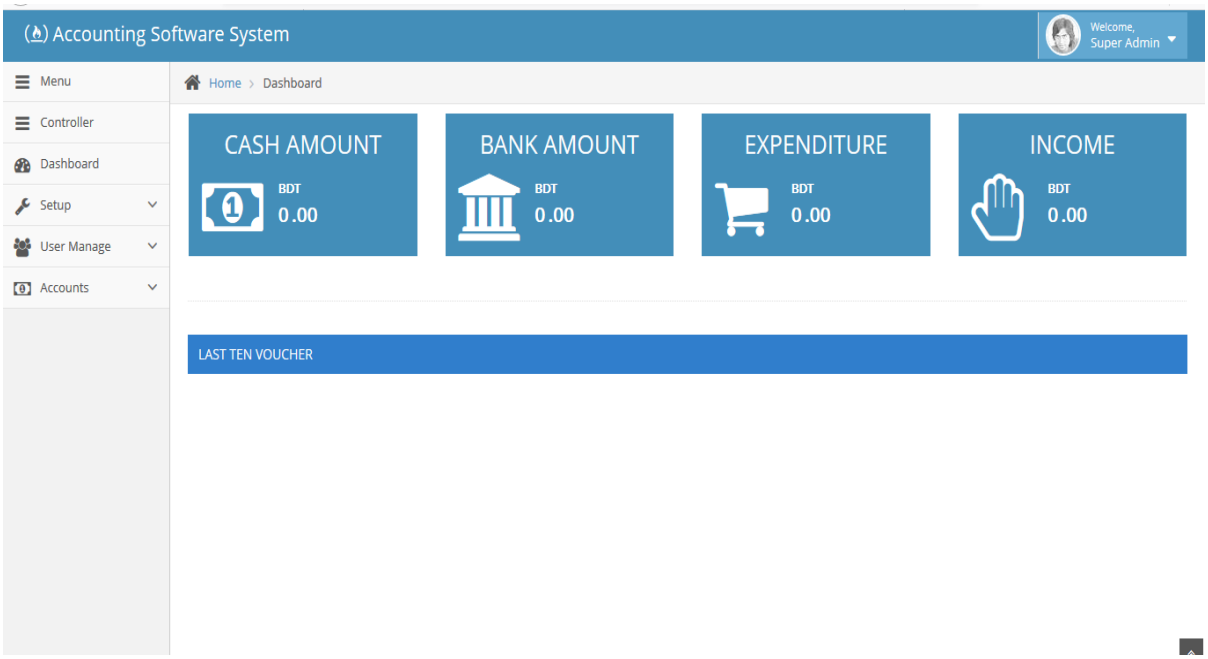



Fig 6.1: Dashboard of Accounting Management System

## 6.2 Update Company information

We can update company information any time when we have need.

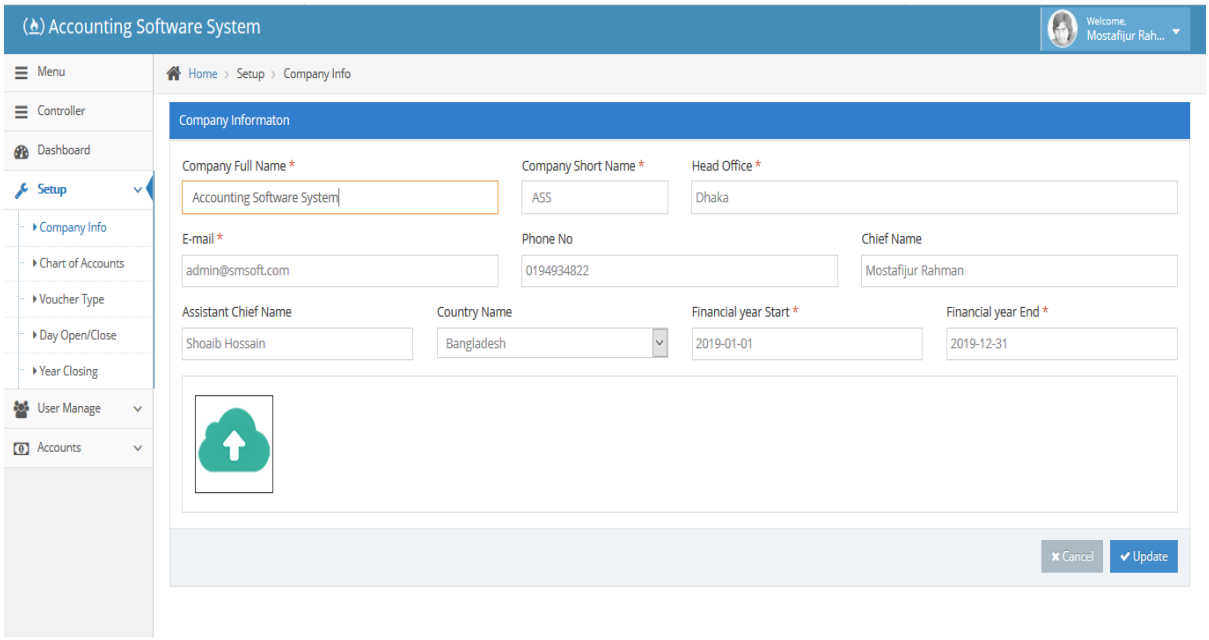



Fig 6.2: Update Company Information

## 6.3 Create User

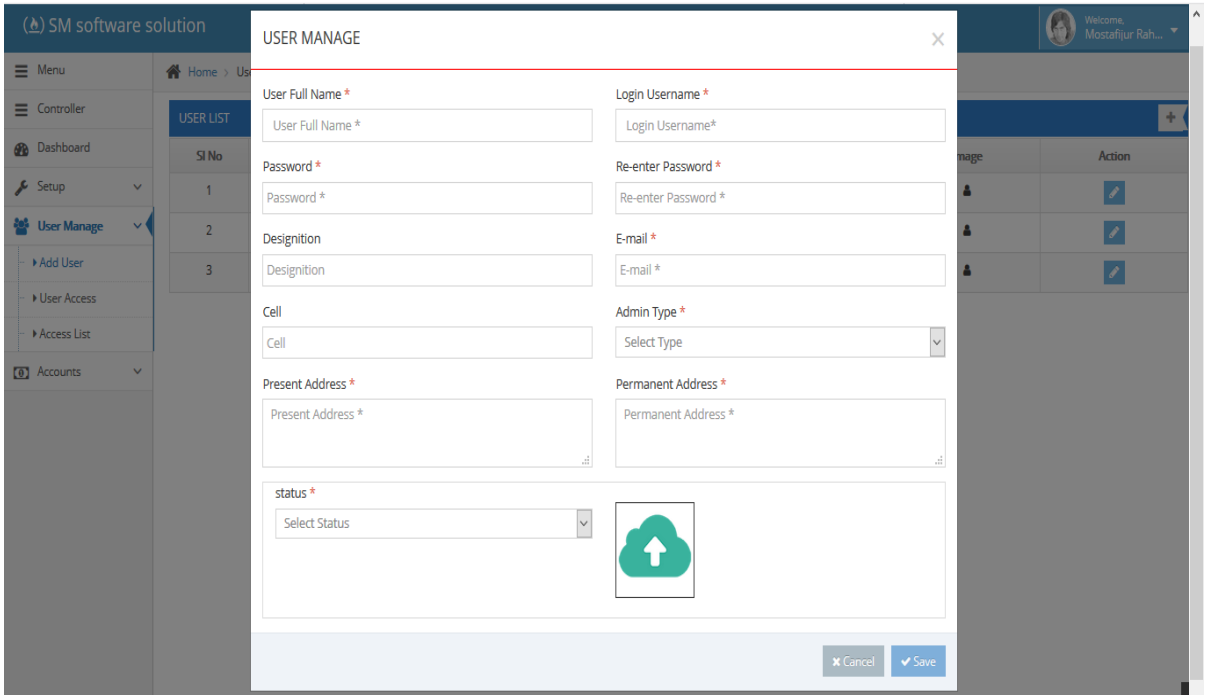



Fig 6.3: Create user

We can create user for the accounting system software manage.

## 6.4 User access manage

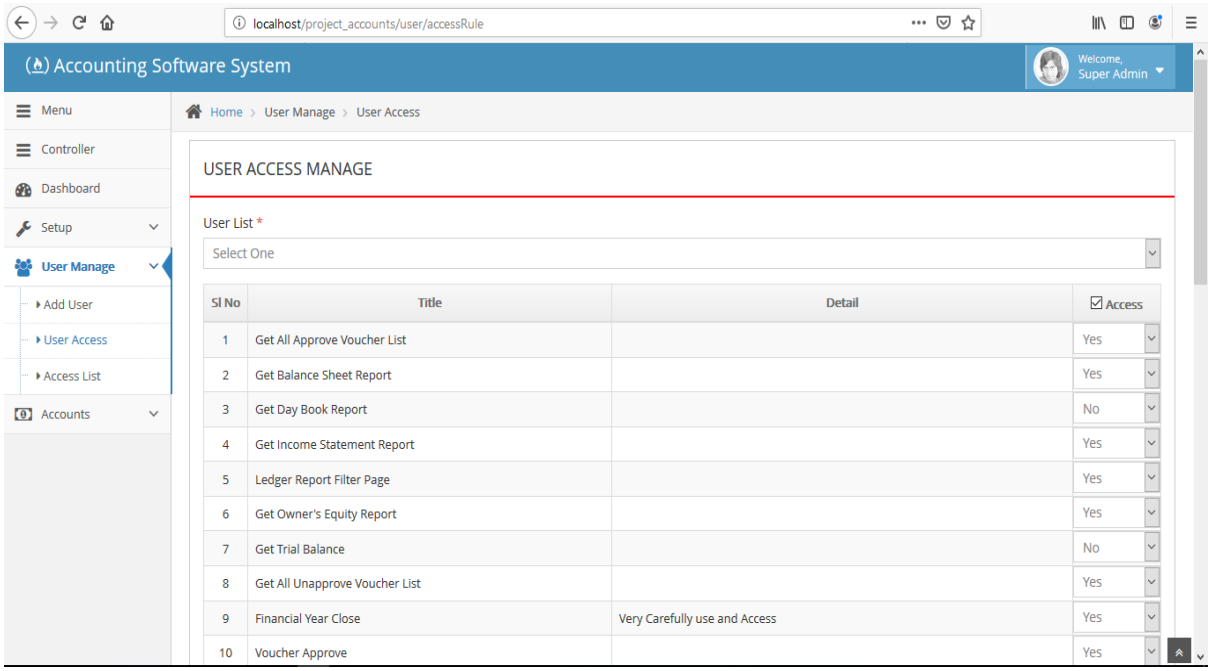



Fig 6.4: User access manage

We can manage user access in this software for these reasons they can't access all section in this software.

## 6.5 User list

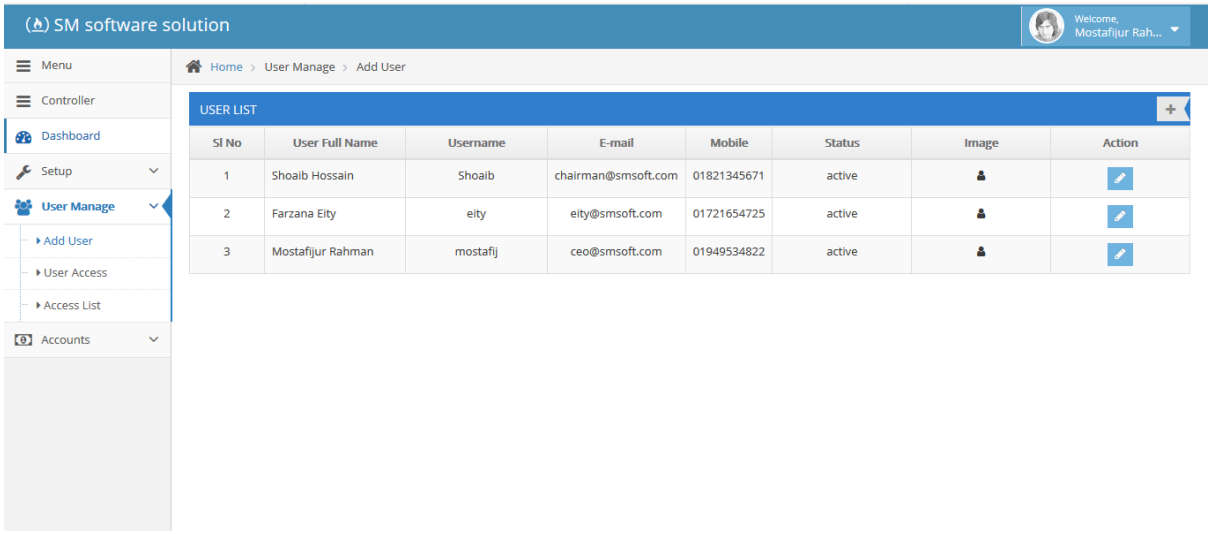



Fig 6.5: User list

We can see user list and also can update user information from there.

## 6.6 Chart of accounts

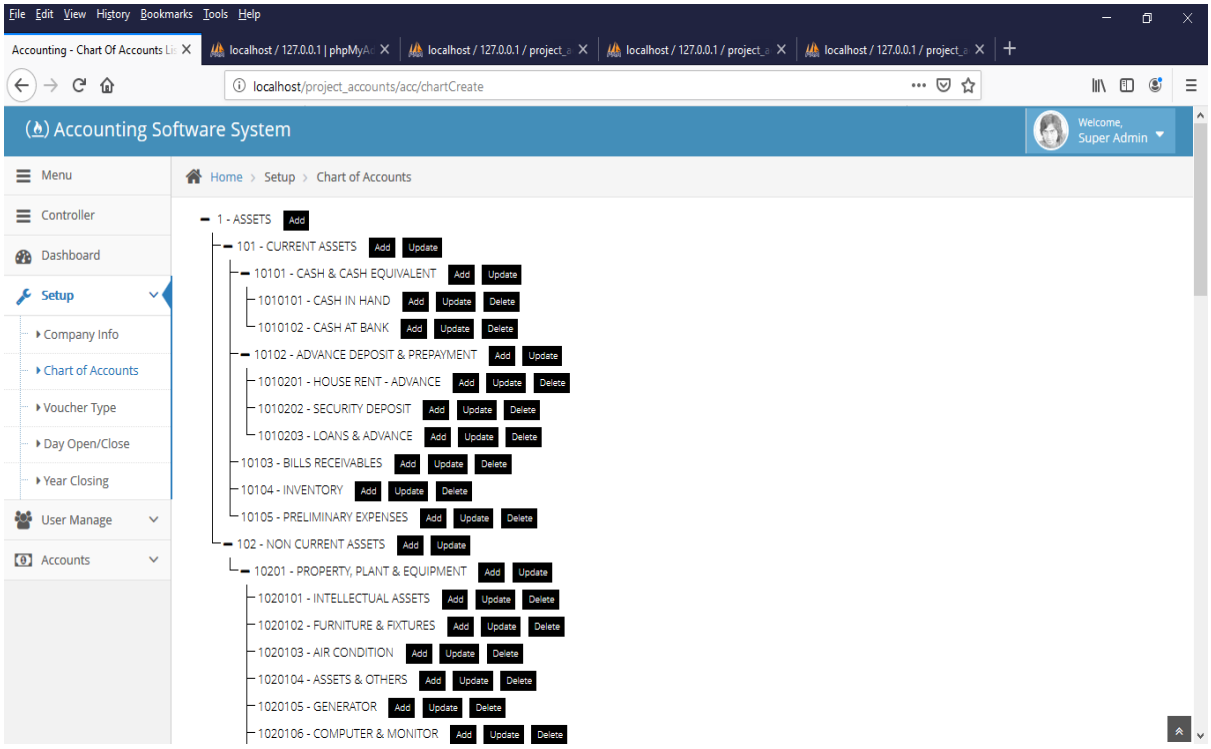



Fig 6.6: chart of accounts

We can see the chart of accounts and also we can add/update/delete Assets, Liabilities, Income and Expenses head in the chart of accounts.

## 6.7 Journal

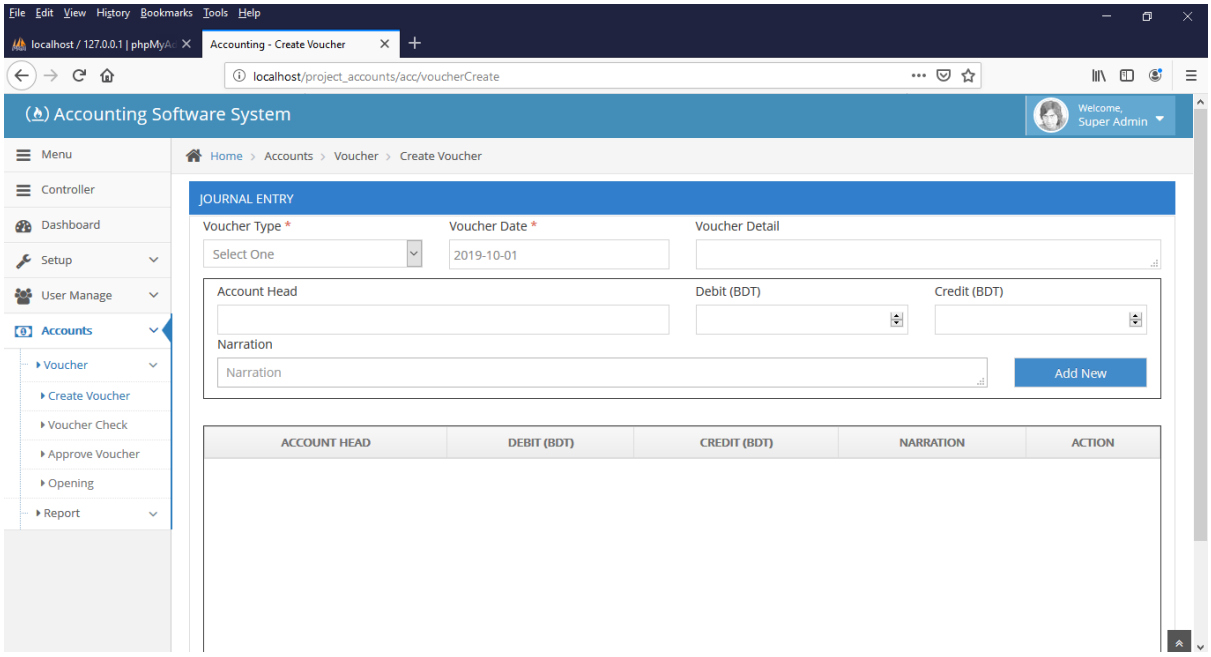



Fig 6.7: Journal Entry

## 6.8 Voucher

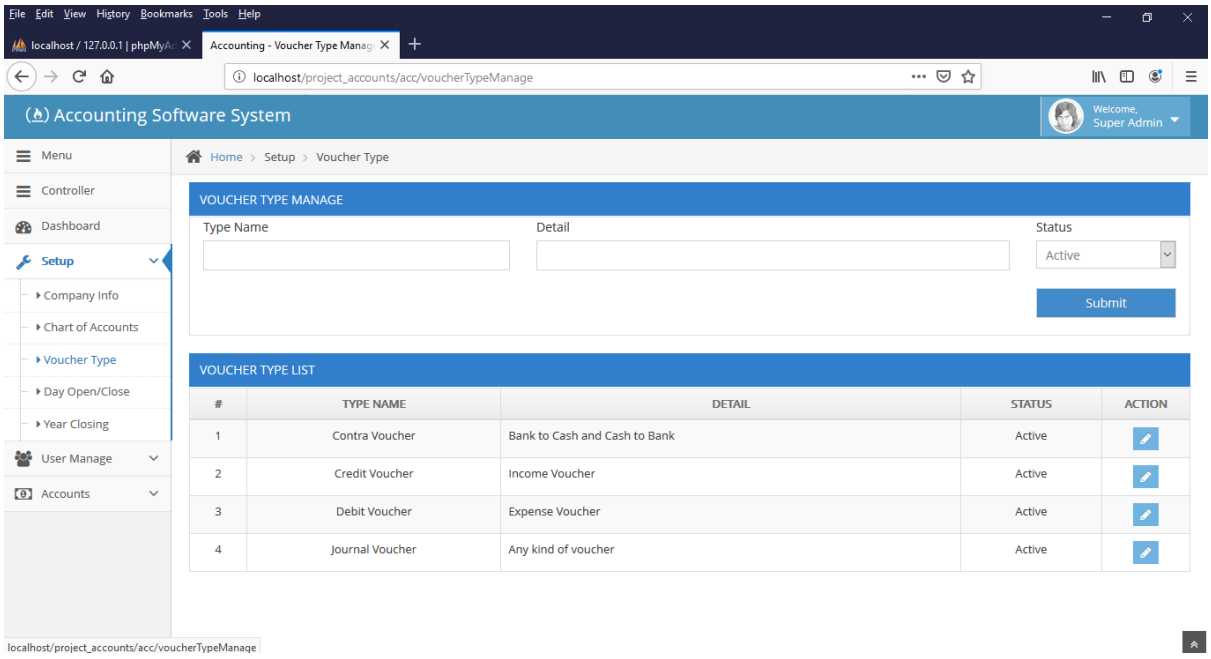



Fig 6.8: Voucher type manage

From there we can see voucher list and also we can create voucher type and manage vouchers also.

# Chapter 7

## TESTING

## 7.1 Software Testing

Testing is a set of activities that can be planned in advance and conducted systematically. Testing is the process of exercising a program with the specific intent of finding errors prior to delivery to the end user. This area is concerned with verifying that a correct solution to the problem, embodied in the statement of the requirements, has been developed. Testing is a multistage process that consists of activities for validating the software product, from the most primitive elements up to the fully integrated system.

## We have done the following testing:

- ✓ Unit Testing
- ✓ Validation Testing
- ✓ System testing
- ✓ Performance Testing
- ✓ Integration Testing

## 7.2 Characteristics of Good Test

An excellent test case satisfies the following criteria:

- ➢ It has a reasonable probability of catching an error you test to find errors. When searching for ideas for test cases, try working backwards from an idea of how the program might fail. If the program could fail in this way, how could you catch it?
- ➢ It is not redundant if two tests look for same error, why run both?
- ➢ It's the best of its breed in a group of similar tests, one can be more effective than the others; you want the best of the breed, the one most likely to find the error.
- ➢ It is neither too simple nor too complex

You can save testing time by combining two or more tests into one test case. But don't create a monster that's too complicated to execute or understand or that takes too much time to create. It's often more efficient to run simpler tests. Be cautious when combining invalid inputs.

## 7.3 Black box Testing

The technique of testing without having any knowledge of the interior working of the application is Black Box testing. The tester is oblivious to the system architecture and does not have access to the source code. Typically, when performing a black box test, a tester will interact with the system's user interface by providing inputs and examining outputs without knowing how and where the inputs are works upon.

## 7.4 White Box Testing

White box testing is the detailed investigation of internal logic and structure of the code. White box testing is also called glass testing or open box testing. In order to perform

white box testing on an application, the tester needs to possess knowledge of the internal working of the code. The tester needs to have a look inside the source code and find out which unit/chunk of the code is behaving inappropriately.

## 7.5 Grey Box Testing

Grey box testing is a technique to test the application with limited knowledge of the internal working of an application. In software testing, the term the more you know the batter carries a lot of weight when testing an application. Mastering the domain of a system always gives the tester an edge over someone with limited domain knowledge. Unlike black box testing, where the tester only tests the application's user interface, in grey box testing, the tester has access to design documents and the database. Having this knowledge, the tester is able to better prepare test data and test scenarios when making the test plan.

## 7.6 Unit Testing

In computer programming, unit testing is a method by which individual units of source code, sets of one or more computer program modules together with associated control data, usage procedures, and Operating procedures, are tested to determine if they are fit for use. Intuitively, one can view a unit as the smallest testable part of an application. In procedural programming a unit could be an entire module but is more commonly an individual function or procedure. In object-oriented programming a unit is often an entire interface, such as a class, but could be an individual method. Unit tests are created by programmers or occasionally by white box testers during the development process .Ideally, each test case is independent from the others: substitutes like method stubs, mock objects, fakes and test harnesses can be used to assist testing a module in isolation. Unit tests are typically written and run by software developers to ensure that code meets its design and behaves as intended.

## 7.7 Integration Testing

Integration testing is a logical extension of unit testing. In its simplest form, two units that have already been tested are combined into a component and the interface between them is tested. A component, in this sense, refers to an integrated aggregate of more than one unit. In a realistic scenario, many units are combined into components, which are in turn aggregated into even larger parts of the program. The idea is to test combinations of pieces and eventually expand the process to test your modules with those of other groups. Eventually all the modules making up a process are tested together. Beyond that, if the program is composed of more than one process, they should be tested in pairs rather than all at once.

Integration Testing identifies problems that occur when units are combined. By using a test plan that requires you to test each unit and ensure you the viability of each before combining units, you know that any errors discovered when combining units are likely

related to the interface between units. This meth method reduces the number of possibilities to a far simpler level of analysis.

## 7.8 The Nature of Software Defects

Logic errors and incorrect assumptions are inversely proportional to the probability that a program path will be executed. General processing tends to be well understood while special case processing tends to be prone to errors. We often believe that a logical path is not likely to be executed when it may be executed on a regular basis. Our unconscious assumptions about control flow and data lead to design errors that can only be detected by path testing. Typographical errors are random.

### 7.8.1 Basis Path Testing

This method enables the designer to derive a logical complexity measure of a procedural design and use it as a guide for defining a basis set of execution paths. Test cases that exercise the basis set are guaranteed to execute every statement in the program at least once during testing.

#### 7.8.1.1 Flow Graphs

Flow graphs can be used to represent control flow in a program and can help in the derivation of the basis set. Each flow graph node represents one or more procedural statements. The edges between nodes represent flow of control. An edge must terminate at a node, even if the node does not represent any useful procedural statements.

A region in a flow graph is an area bounded by edges and nodes. Each node that contains a
condition is called a predicate node. Cyclamate complexity is a metric that provides a quantitative measure of the logical complexity of a program. It defines the number of independent paths in the basis set and thus provides an upper bound for the number of tests that must be performed.

### 7.8.2 The Basis Set

An independent path is any path through a program that introduces at least one new set of processing statements (must move along at least one new edge in the path). The basis set is not unique. Any number of different basis sets can be derived for a given procedural design

#### 7.8.2.1 Automating Basis Set Derivation

The derivation of the flow graph and the set of basis paths are amenable to automation. A software tool to do this can be developed using a data structure called a graph matrix. A

graph matrix is a square matrix whose size is equivalent to the number of nodes in the flow graph.
Each row and column corresponds to a particular node and the matrix corresponds to the connections (edges) between nodes. By adding a link weight to each matrix entry, more information about the control flow can be captured. In its simplest form, the link weight is 1 if an edge exists and 0 if it does not. But other types of link weights can be represented:

- The probability that an edge will be executed,
- The processing time expended during link traversal,
- The memory required during link traversal, or
- The resources required during link traversal

Graph theory algorithms can be applied to these graph matrices to help in the analysis necessary
to produce the basis set.

### 7.8.3 Loop Testing

This white box technique focuses exclusively on the validity of loop constructs. Four different
classes of loops can be defined:

- Simple loops,
- Nested loops,
- Concatenated loops, and
- Unstructured loops.

#### 7.8.3.1 Simple Loops

The following tests should be applied to simple loops where *n* is the maximum number of allowable passes through the loop:

- Skip the loop entirely,
- Only pass once through the loop,
- *m* passes through the loop where *m* <*n*,
- *n – 1, n, n + 1* passes through the loop.

#### 7.8.3.2 Nested Loops

The testing of nested loops cannot simply extend the technique of simple loops since this would result in a geometrically increasing number of test cases. One approach for nested loops:

- Start at the innermost loop. Set all other loops to minimum values.
- Conduct simple loop tests for the innermost loop while holding the outer loops at their minimums. Add tests for out-of-range or excluded values.

- Work outward, conducting tests for the next loop while keeping all other outer loops at
  minimums and other nested loops to *typical* values.
- Continue until all loops have been tested.

### 7.8.3.3 Concatenated Loops

Concatenated loops can be tested as simple loops if each loop is *independent* of the others. If they are not independent (e.g. the loop counter for one is the loop counter for the other), then the nested approach can be used.

### 7.8.3.4 Unstructured Loops

This type of loop should be redesigned not tested!!!

# Chapter 8

## FUTURE SCOPE AND CONCLUSION

## 8.1 Future Scope

In future we are included some of features such as

- ✓ Public Platform
- ✓ Budget
- ✓ Inventory
- ✓ HR
- ✓ CSV upload of all possible case
- ✓ Excel export of all possible case
- ✓ Modify user Access.

## 8.2 Conclusion

This project's aim was to develop an automated system for managing accounts activities. It provides users with a friendly interface and easy access to the database. It saves time and makes records easier to manage. This system provides user authentication facility, which makes this system more reliable to the organization. It also a record has adequate functionality to run any accounts managements system.

Now words any business must needed accounting software to increase business productivity. Accounting software helps to evaluate business performance that shows whether the business is improving or depreciating. It ensures effective cash flow management. Accounting software allows making right business decisions as it gives a clear picture of the business. Business persons can reduce debt burdens and fulfill tax requirements of the business using it.

This report has been completed in fulfillment for the partial requirements of the course CSE 477: Project/Thesis. The work done in order to complete this report has been highly useful. It has taught about the different development phases of software development life cycle. After completing this project, we have come to know that real life experience gives the opportunity to increase our knowledge in the area of software development which will certainly be helpful for our career.

# References

[1] Software Engineering, A Practitioner's Approach, Seventh Edition Roger S. Pressman.

[2] Software Engineering, Eight Edition or later, Ian Somerville.

[3] Database Management System Concept.

[4] https://www.tutorialspoint.com/sdlc/sdlc_waterfall_model.html

[5] https://en.wikipedia.org/wiki/PHP

[6] https://en.wikipedia.org/wiki/JavaScript

[7] https://jquery.com/

[8] https://en.wikipedia.org/wiki/Cascading_Style_Sheets

[9] https://en.wikipedia.org/wiki/MySQL

[10] https://www.codeigniter.com/